



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Algorithm:

1. convert array into heap $\begin{cases} \text{max (Ascending sort)} \\ \text{min (Descending sort)} \end{cases}$

2. delete all elements from ^{heap} one by one and put them on empty locations from right side.

to convert into heap : $n \log n$
to delete from heap : $n \log n$

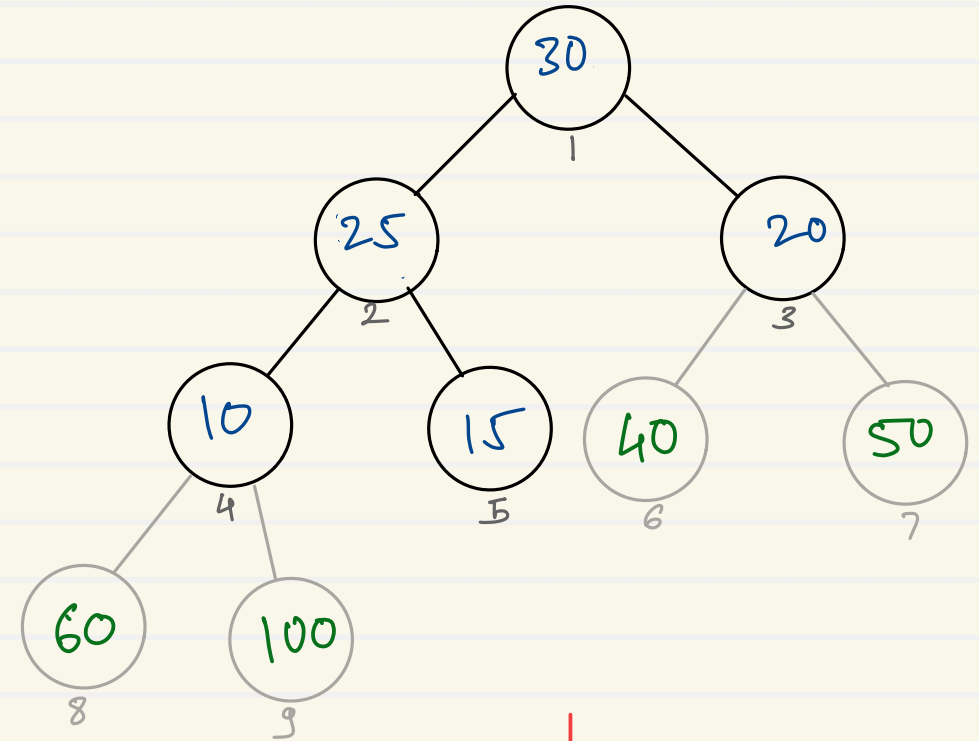
Time $\propto 2n \log n$

$$T(n) = O(n \log n)$$

Best
Avg
Worst

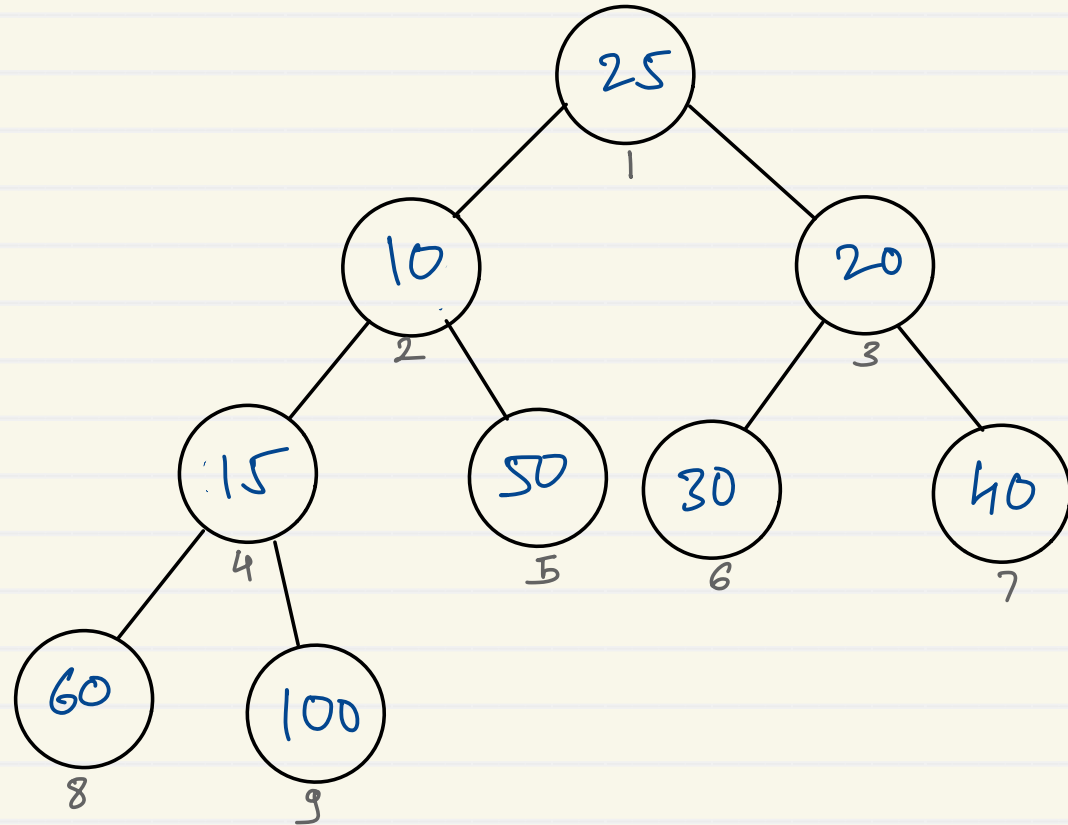
$$S(n) = O(1)$$

25	10	20	15	50	30	40	60	100
1	2	3	4	5	6	7	8	9

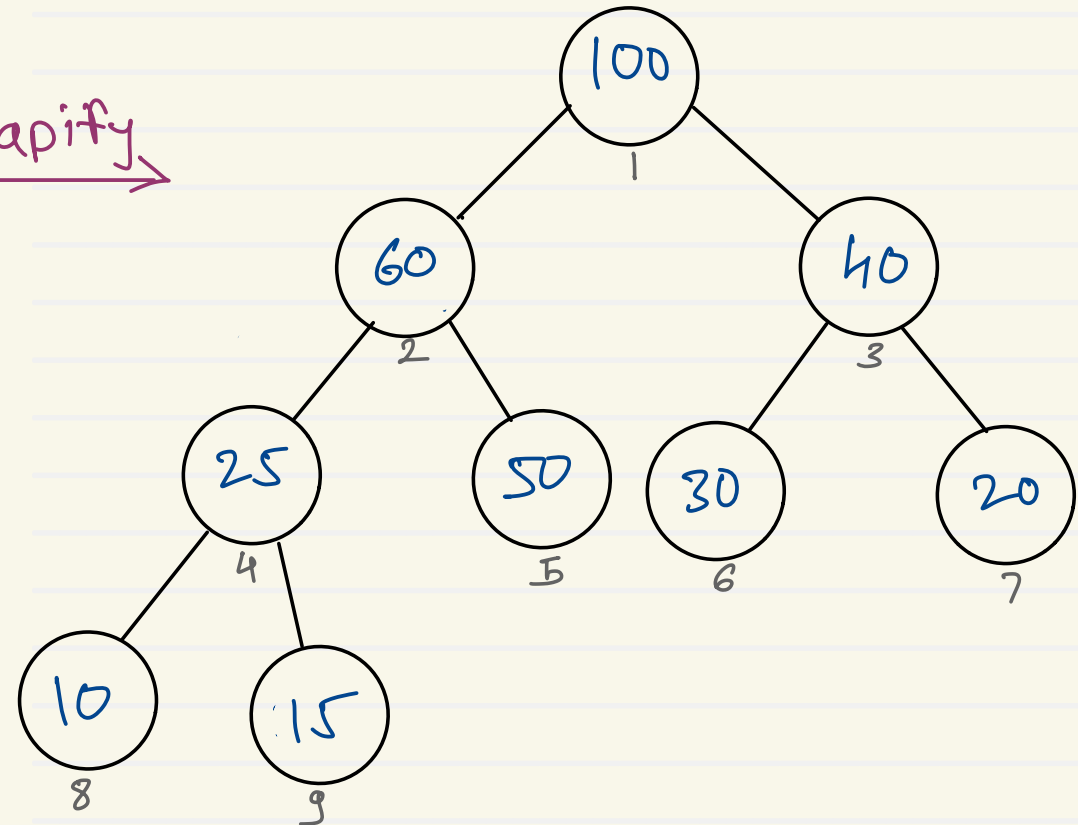


30	25	20	10	15	40	50	60	100
1	2	3	4	5	6	7	8	9

Heapify (to convert array into heap (max/min)) $\rightarrow T(n) = O(n)$



heapify $\rightarrow$



25	10	20	15	50	30	40	60	100
1	2	3	4	5	6	7	8	9

25	100	40	60	50	30	20	10	15
1	2	3	4	5	6	7	8	9

Merge sort

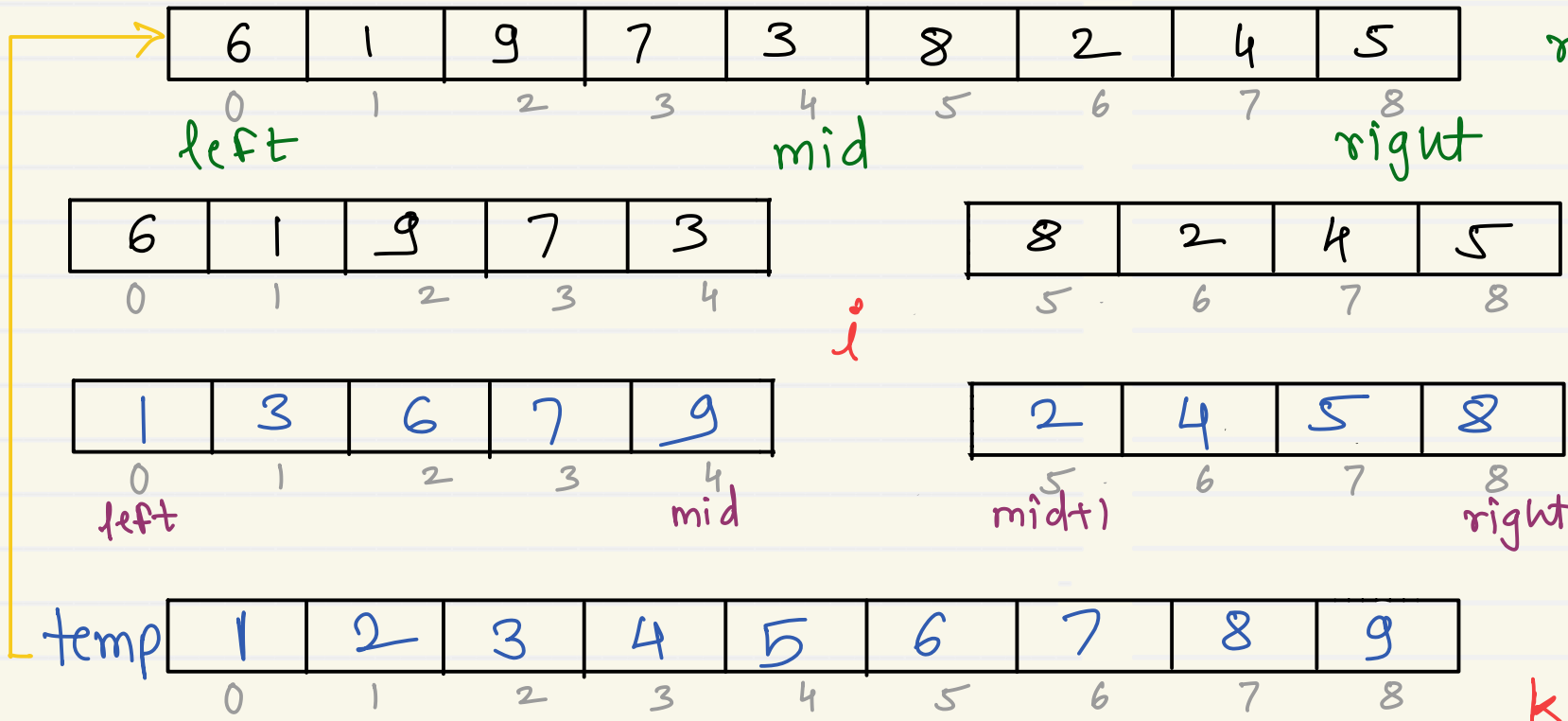
1. Divide array in two parts
2. Sort both partitions individually (by merge sort only)
3. Merge sorted partitions into temporary array
4. Overwrite temporary array into original array

$$mid = \frac{left + right}{2}$$

left partition: left to mid
right partition: mid+1 to right

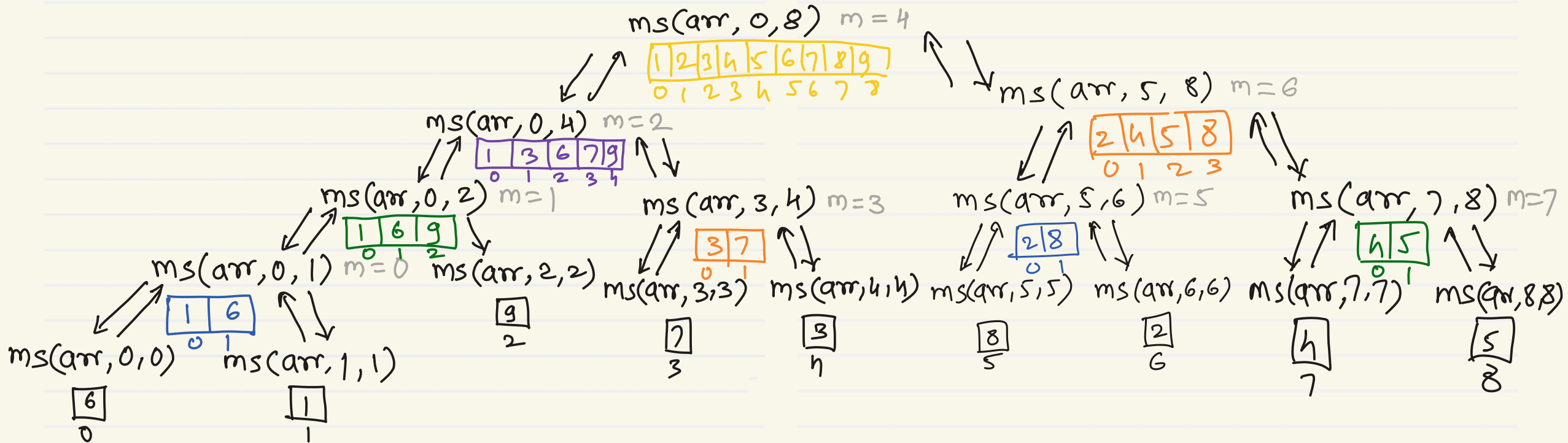
$$sizeof(temp) = right - left + 1$$

$i = left$ to mid
 $j = mid+1$ to $right$
 $k = 0$ to $size-1$



Merge sort

1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9
6	1	8	7	3	8	2	4	5
0	1	2	3	4	5	6	7	8



Merge sort

Number of elements = n

levels of division = $\log n$

comps per level = n

Total comps = $n \log n$

Time $\propto$ comps

Time $\propto n \log n$

Best
Avg
Worst

$$T(n) = O(n \log n)$$

temp $\rightarrow$ extra space needed to merge two sorted partitions.

Auxiliary space $\propto n$

$$S(n) = O(n)$$

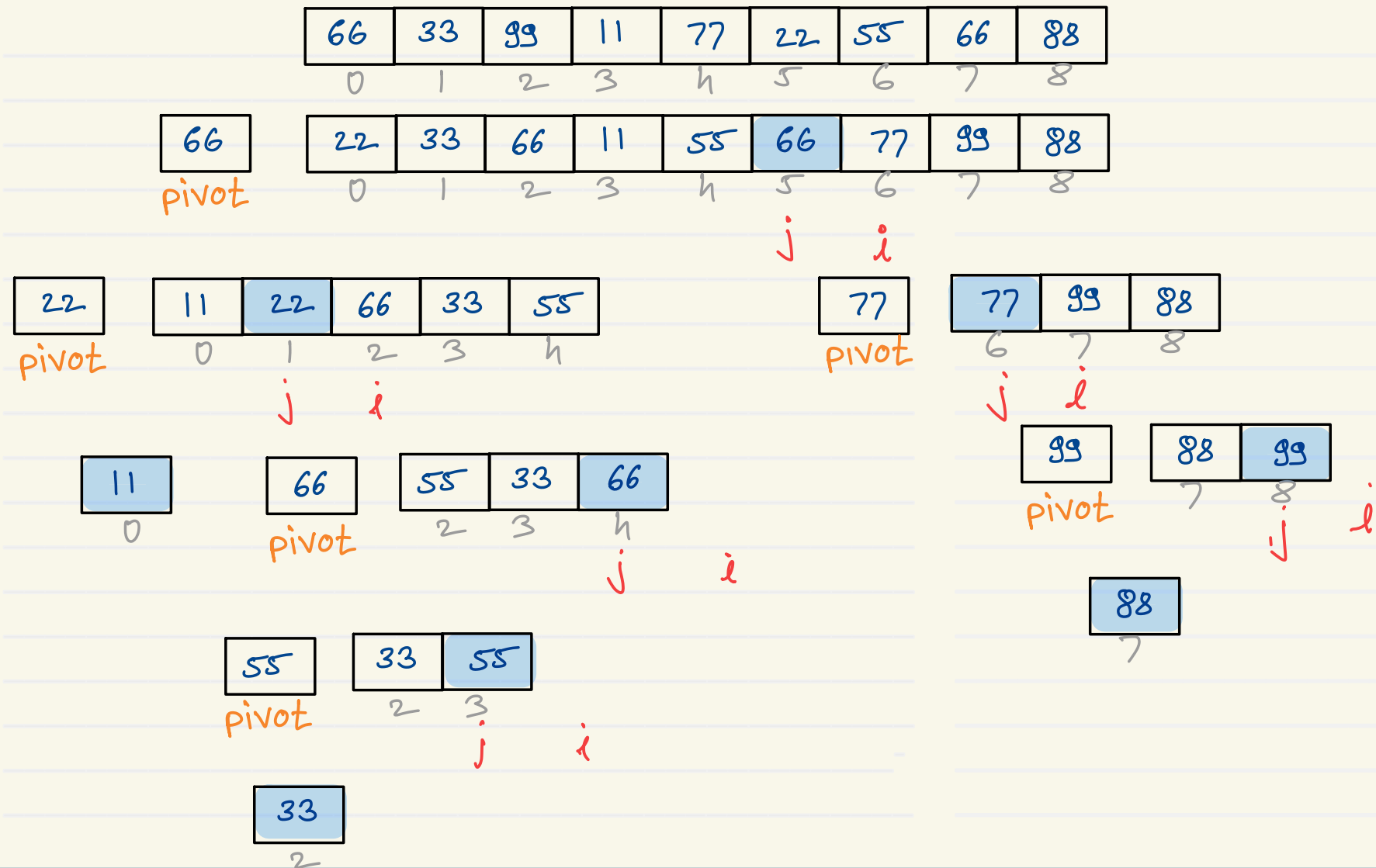
- merge sort is out place sorting algo

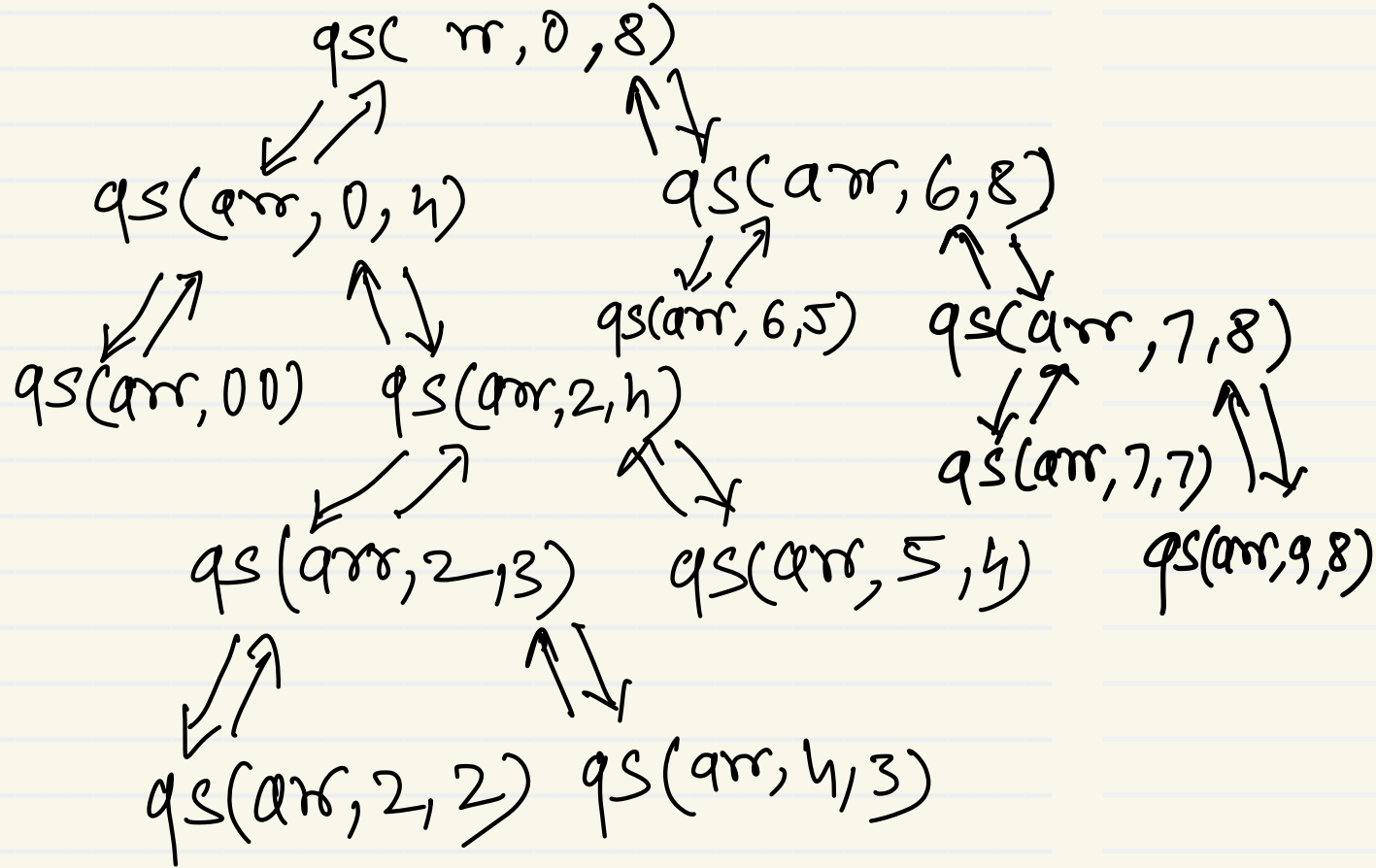
1. Select pivot/axis/reference element from array
2. Arrange lesser elements on left side of pivot
3. Arrange greater elements on right side of pivot
4. Sort left and right side of pivot again (by quick sort)

Selection pivot :

1. extreme left or right
2. middle element
3. median
 - random 3 element
 - random 5 element
 - random 7 element
4. dual pivot

Quick sort





Quick sort

Number of elements = n

levels of division = $\log n$

comps per level = n

Total comps = $n \log n$

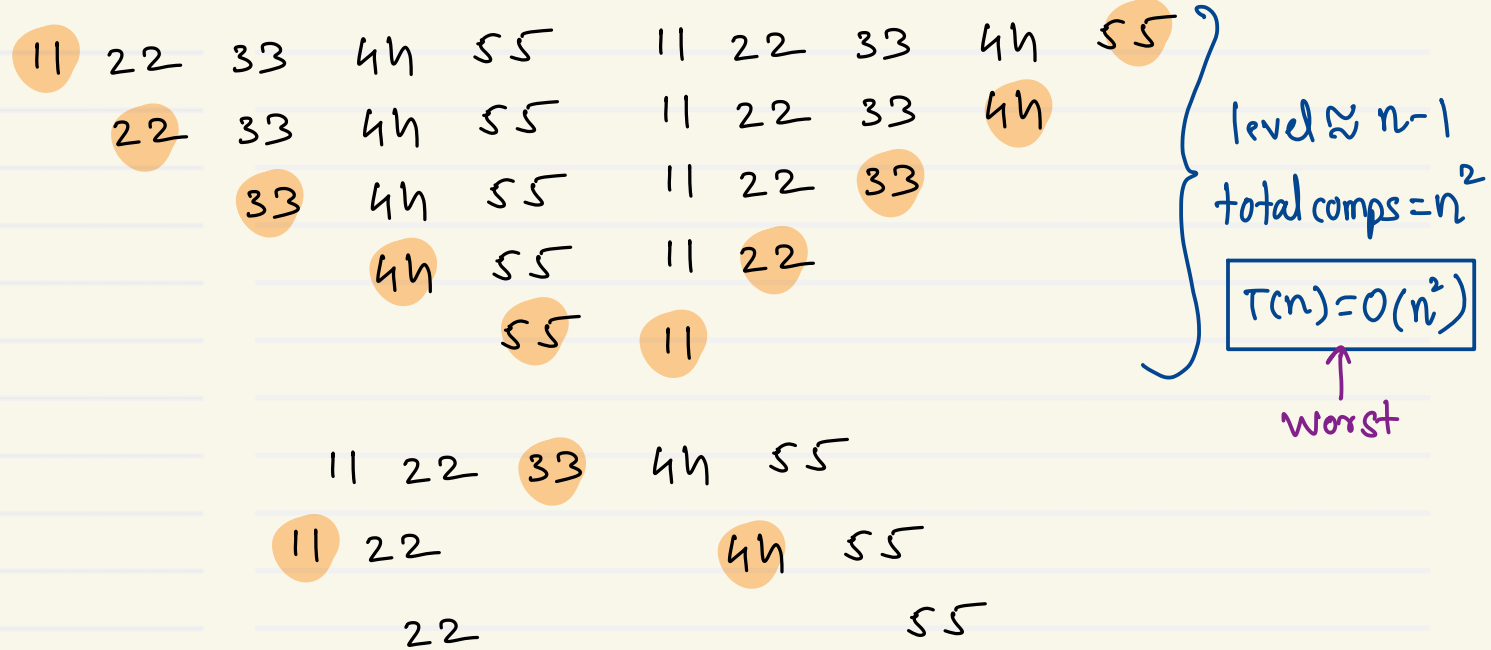
Time $\propto$ comps

Time $\propto n \log n$

Best
Avg

$$T(n) = O(n \log n)$$

$$S(n) = O(1)$$



- in quick sort time complexity is dependent on selection pivot.